INVISIBLE CLOAK

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

NANDINI SINGH

EN23CS301658

Under the Guidance of

Prof. SONAL MODH

Prof. RASHMI CHAUDHARY

MEDICAPS UNIVERSITY

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

INVISIBLE CLOAK

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

NANDINI SINGH

EN23CS301658

Under the Guidance of

Prof. SONAL MODH

Prof. RASHMI CHAUDHARY

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work **“INVISIBLE CLOAK”** is hereby approved as a creditable study of an engineering subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Name: Prof. Sonal Modh and Prof. Rashmi Chaudhary

Designation: Assistant Professor

Affiliation: Medicaps University

External Examiner

Name:

Designation:

Affiliation:

Declaration

I/We hereby declare that the project entitled **"Invisible Cloak"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology in '**CSE Department'** completed under the supervision of **Prof. Sonal Modh and Prof. Rashmi Chaudhary**, **Professor and department of CSE Department ,** Faculty of Engineering, Medi-Caps University Indore is an authentic work.

Further, I declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Certificate

I/We, **Prof. Sonal Modh and Prof. Rashmi Chaudhary** certify that the project entitled **“Invisible Cloak”** submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Nandini Singh** is the record carried out by him/them under my/our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Sonal Modh and Prof. Rashmi Chaudhary

CSE DEPARTMENT

Medicaps University, Indore

External Guide ……………………..

CSE DEPARTMENT

Medicaps University, Indore

Prof (Dr.) Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal,** who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof.(Dr.)D.K. Patnaik,** Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.)Ratnesh Litoriya,** Dean, Faculty of Engineering, Medi-Caps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Prof. (Dr.) Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Nandini Singh
B.Tech. III Year (K)
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Abstract

The *Invisible Cloak Project* is a real-time computer vision application developed using Python and OpenCV. The objective of this project is to create an invisibility effect by detecting a specific color (pink) in a live video stream and replacing it with a pre-captured background. The system uses image processing techniques such as color detection in HSV color space, masking, and morphological operations to achieve smooth and accurate results.

The project demonstrates how basic computer vision concepts can be applied to create visually appealing effects. It also provides practical understanding of real-time video processing and image segmentation. This system can be further extended to advanced applications such as augmented reality, video editing, and interactive systems.

Keywords: Computer Vision, OpenCV, HSV, Image Processing, Invisibility Cloak

Table of Contents

List of Figures

List of Tables

Abbreviations

AI – Artificial Intelligence

CV – Computer Vision

HSV – Hue, Saturation, Value

RGB – Red, Green, Blue

BGR – Blue, Green, Red

IDE – Integrated Development Environment

DFD – Data Flow Diagram

ER – Entity Relationship

UI – User Interface

RAM – Random Access Memory

FPS – Frames Per Second

Notations & Symbols

III → Input Image Frame

BBB → Background Frame

H,S,VH, S, VH,S,V → Hue, Saturation, Value components

MMM → Mask Image

OOO → Output Frame

f(x,y)f(x, y)f(x,y) → Pixel value at coordinate (x, y)

∩ → Intersection operation (masking)

∪ → Union operation (combining frames)

Chapter-1

Introduction

1.1 Introduction

Computer Vision is a field of artificial intelligence that enables machines to interpret and process visual data from images and videos. It plays a major role in modern applications such as facial recognition, object detection, and augmented reality.

The Invisible Cloak Project is a simple yet effective implementation of computer vision techniques. It works by capturing a background frame and detecting a specific color (pink) in real-time video. The detected region is then replaced with the background, creating an illusion of invisibility.

1.2 Literature Review

Several techniques have been developed for image processing and video manipulation. Traditional chroma keying (green screen) is widely used in films to replace backgrounds.

Research shows that HSV color space is more effective than RGB for color detection due to its separation of color and intensity. Morphological operations such as dilation and erosion are used to reduce noise in image processing.

OpenCV provides efficient tools for implementing these techniques in real-time applications. This project uses these established methods in a simplified form.

1.3 Objectives

- To develop a real-time invisibility cloak system
- To detect a specific color using HSV color space
- To apply masking and background replacement
- To understand real-time image processing techniques

1.4 Significance

This project demonstrates the practical application of computer vision in creating visual effects. It helps in understanding key concepts such as color detection, masking, and real-time video processing.

The significance of this project lies in its ability to bridge theoretical knowledge with practical implementation. It also provides a foundation for future advancements in fields like augmented reality, surveillance systems, and automated vision-based applications.

1.5 Research Design

The project follows a **practical implementation approach** using Python and OpenCV. It includes capturing data (video frames), processing it using algorithms, and generating output in real-time.

1.6 Source of Data

The primary source of data for this project is real-time video captured using a webcam. The system processes live frames continuously to detect the specified color and apply the invisibility effect.

Additionally, a background frame is captured at the beginning and used throughout the processing phase for replacement purposes.No external datasets or databases are required for this project.

1.7 Chapter Scheme

The report is organized into the following chapters:

- **Chapter 1: Introduction** – Provides an overview, objectives, and significance of the project
- **Chapter 2: Analysis Model** – Includes use case diagram, DFD, and system analysis
- **Chapter 3: Design Model** – Describes system architecture, components, and design details
- **Chapter 4: Implementation** – Explains coding and tools used
- **Chapter 5: Results and Discussion** – Shows outputs and performance analysis

- **Chapter 6: Conclusion and Future Scope** – Summarizes findings and suggests improvements

Chapter-2

Requirements Specification

2.1 User Characteristics

The intended users of the system are students, beginners in computer vision, and individuals with basic knowledge of computers. The user is expected to have minimal technical expertise, such as the ability to run Python programs and interact with a simple interface.

The system does not require advanced programming knowledge for operation. However, for development or modification, familiarity with Python and OpenCV is beneficial. Users should also be able to understand basic instructions for camera positioning and lighting conditions.

2.2 Functional Requirements

The functional requirements define what the system should do:

- FR1: The system shall capture real-time video using a webcam.
- FR2: The system shall capture and store a background frame before processing begins.
- FR3: The system shall convert video frames from BGR to HSV color space.
- FR4: The system shall detect a predefined color (pink) in the frame.
- FR5: The system shall generate a mask to isolate the detected color region.
- FR6: The system shall apply morphological operations to reduce noise.
- FR7: The system shall replace the detected region with the stored background.
- FR8: The system shall display the processed video output in real time.
- FR9: The system shall allow the user to terminate execution using a key (e.g., ESC).

2.3 Dependencies

- Python installed
- OpenCV and NumPy libraries
- Working webcam

2.4 Performance Requirements

The system must meet the following performance criteria:

- The system should process video frames in real time with minimal delay.
- The invisibility effect should appear smooth and continuous.
- Color detection should be accurate under normal lighting conditions.
- The system should handle continuous video streaming without crashing.

2.5 Hardware Requirements

- Webcam
- Minimum 4GB RAM
- Processor: i3 or above

2.6 Constraints & Assumptions

Constraints

- The system works effectively only under stable lighting conditions
- Background must remain static during execution
- Only one color can be detected at a time
- Performance depends on camera quality and system hardware

Assumptions

- The user provides correct color input
- The webcam is properly connected and functioning
- Required software libraries are installed correctly
- The environment is suitable for color detection (no excessive noise or shadows)

Chapter-3

3.1 Algorithm

1. Start webcam
2. Capture background
3. Read video frames
4. Convert to HSV
5. Detect color
6. Create mask
7. Replace region with background
8. Display output

3.2 Function-Oriented Design (Procedural Approach)

Main Functions

- capture_background()
  Captures and stores the background frame
- capture_frame()
  Reads frames continuously from the webcam
- convert_to_hsv(frame)
  Converts BGR image to HSV
- detect_color(hsv_frame)
  Identifies the cloak color
- generate_mask(frame)
  Creates a binary mask
- apply_morphology(mask)
  Removes noise using erosion and dilation
- replace_background(frame, mask)
  Replaces cloak region with background
- display_output(frame)
  Displays the final processed frame

3.3 System Design

The system follows a **pipeline architecture**:
Input → Processing → Output

3.3.1 Data Flow Diagrams

Level 0 DFD

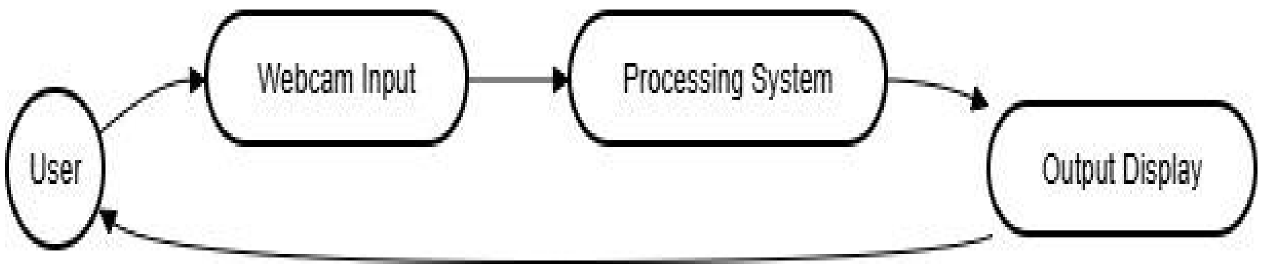


Figure 3.1

Level 1 DFD

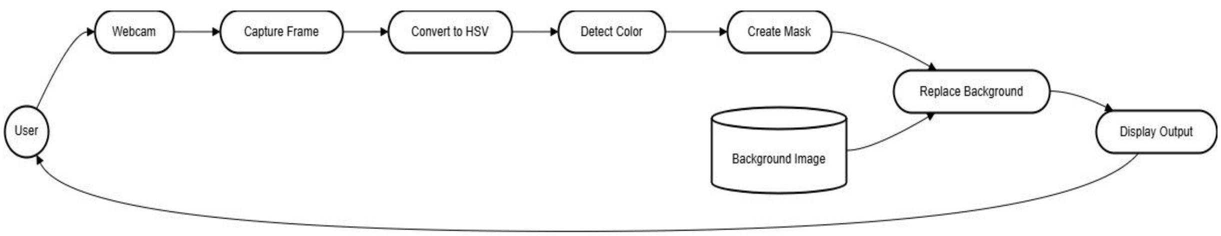


Figure 3.2

3.3.2 Activity Diagram

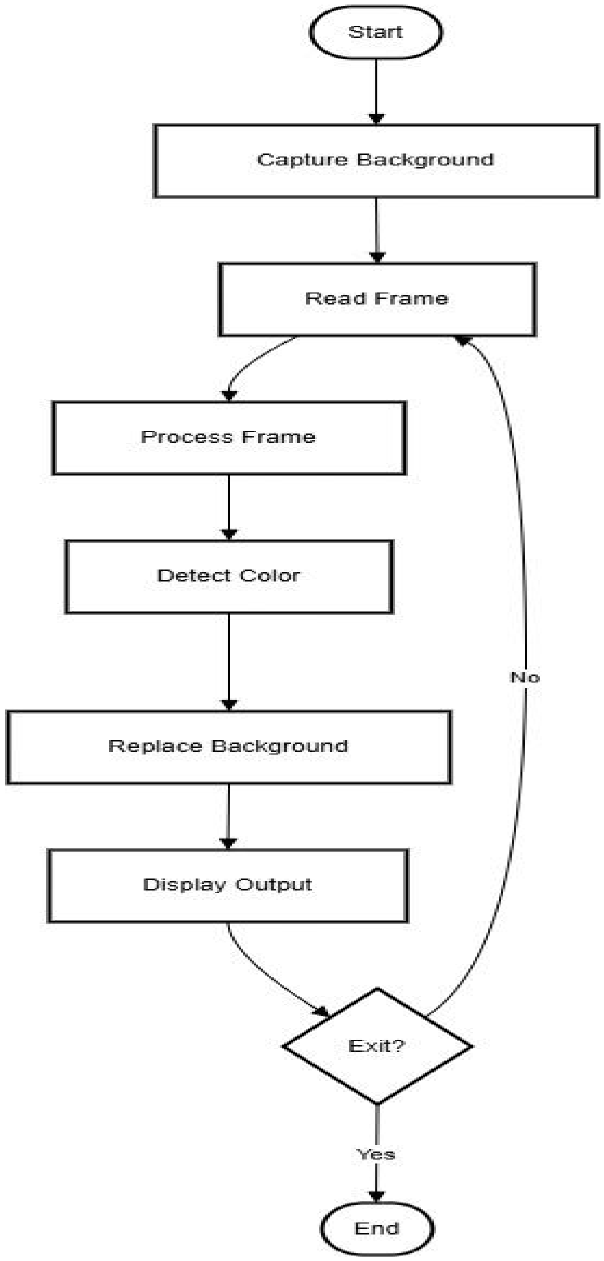

Start
Capture Background
Read Frame
Process Frame
Detect Color
Replace Background
Display Output
Exit?
No
Yes
End

Figure 3.4

3.3.3 Flow Chart

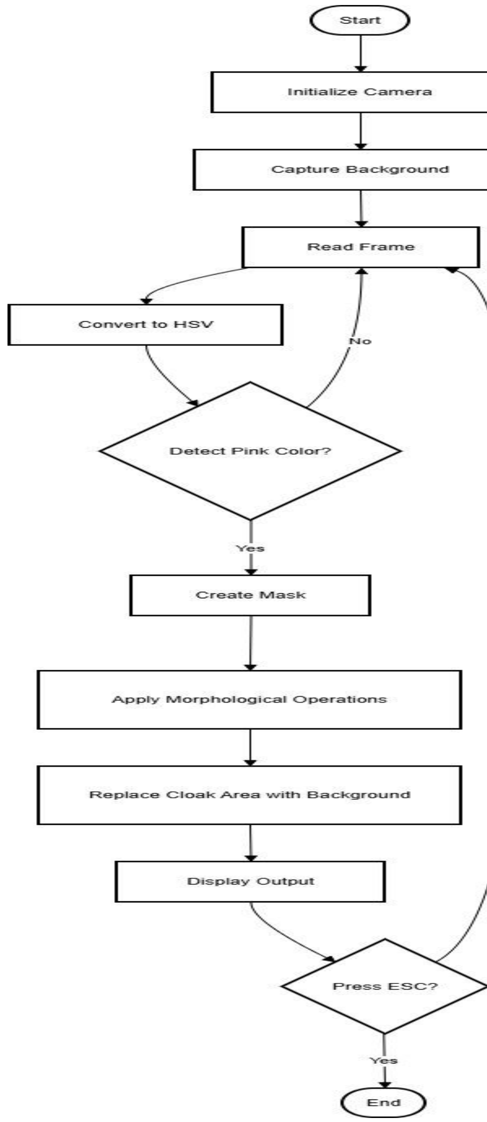


Figure 3.5

3.3.4 Class Diagram

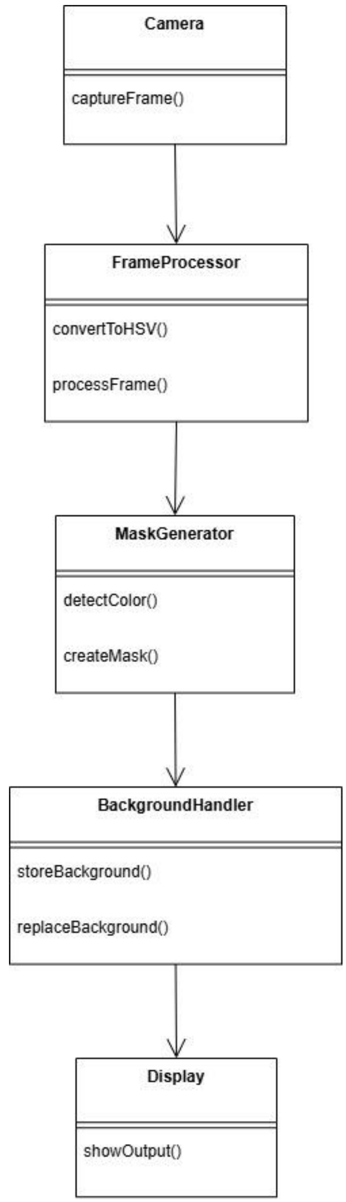


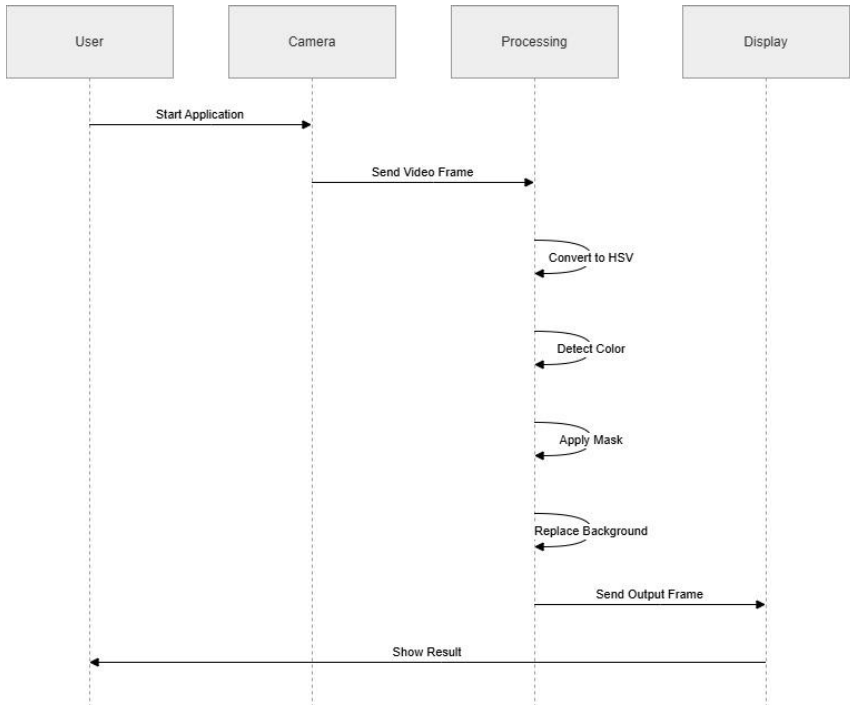
Figure 3.6

3.3.5 ER Diagram

Not applicable because:

- No database is used
- System processes real-time data only

3.3.6 Sequence Diagram



3.4 Database Design

The system does not use a database. All data is processed in real time and not stored permanently.

3.4.1 Logical Database Design

Not applicable due to absence of database.

3.4.2 Physical Database Design

Not applicable since no physical storage system is used.

Chapter-4

4.1 Introduction to Languages, IDEs, Tools and Technologies Used

The Invisible Cloak System is implemented using modern programming tools and libraries that support real-time image processing.

Programming Language

- **Python:**
  Python is used due to its simplicity, readability, and strong support for computer vision libraries.

Libraries Used

- **OpenCV (cv2):**
  Used for image processing tasks such as video capture, color detection, masking, and background replacement.
- **NumPy:**
  Used for efficient numerical operations and handling image arrays.

IDE / Development Environment

- **Visual Studio Code (VS Code):**
  Used for writing and executing Python code. It provides extensions, debugging tools, and an integrated terminal.

Technologies Used

- Computer Vision
- Image Processing
- Real-Time Video Processing

4.2 Testing Techniques and Test Plans

Testing Techniques Used

- **Unit Testing:**
  Individual components such as color detection and masking are tested separately.
- **Integration Testing:**
  Ensures all modules (camera, processing, output) work together correctly.

- **System Testing:**
  The complete system is tested to verify real-time invisibility effect.

Test Plan

Test Case	Input	Expected Output	Result
TC1	Start application	Camera opens	Pass
TC2	Capture background	Background stored	Pass
TC3	Show pink cloth	Cloak becomes invisible	Pass
TC4	No cloak	Normal frame displayed	Pass
TC5	Press ESC	Application closes	Pass

4.3 Installation Instructions

Follow these steps to install and run the project:

1. Install Python (version 3.x)
2. Install required libraries:
   pip install opencv-python numpy
3. Open the project folder in VS Code
4. Run the Python file
5. Ensure webcam is connected and working

4.4 End User Instructions

1. Run the program
2. Allow the system to capture background (stand away initially)
3. Hold a pink-colored cloth/object
4. The system will detect and replace it with background
5. Observe invisibility effect in real time
6. Press **ESC** to exit

Chapter-5

5.1 User Interface Representation

The system provides a simple and minimal user interface based on real-time video display. The interface consists of:

- Camera Window: Displays live video feed
- Processed Output Window: Shows the invisibility effect
- Keyboard Control: User presses a key (ESC) to exit

The interface is designed to be user-friendly and does not require complex interactions. The focus is on real-time visualization rather than graphical controls.

5.2 Brief Description of Various Modules

The system is divided into the following modules:

1. Camera Module

Captures real-time video frames from the webcam. It serves as the input source for the system.

2. Background Capture Module

Captures a static background frame before processing begins. This background is used later for replacement.

3. Frame Processing Module

Converts frames into HSV color space and prepares them for color detection.

4. Color Detection Module

Identifies the specified color (pink) using HSV ranges.

5. Mask Generation Module

Creates a binary mask that isolates the cloak region.

6. Morphological Processing Module

Applies operations like erosion and dilation to remove noise and improve detection accuracy.

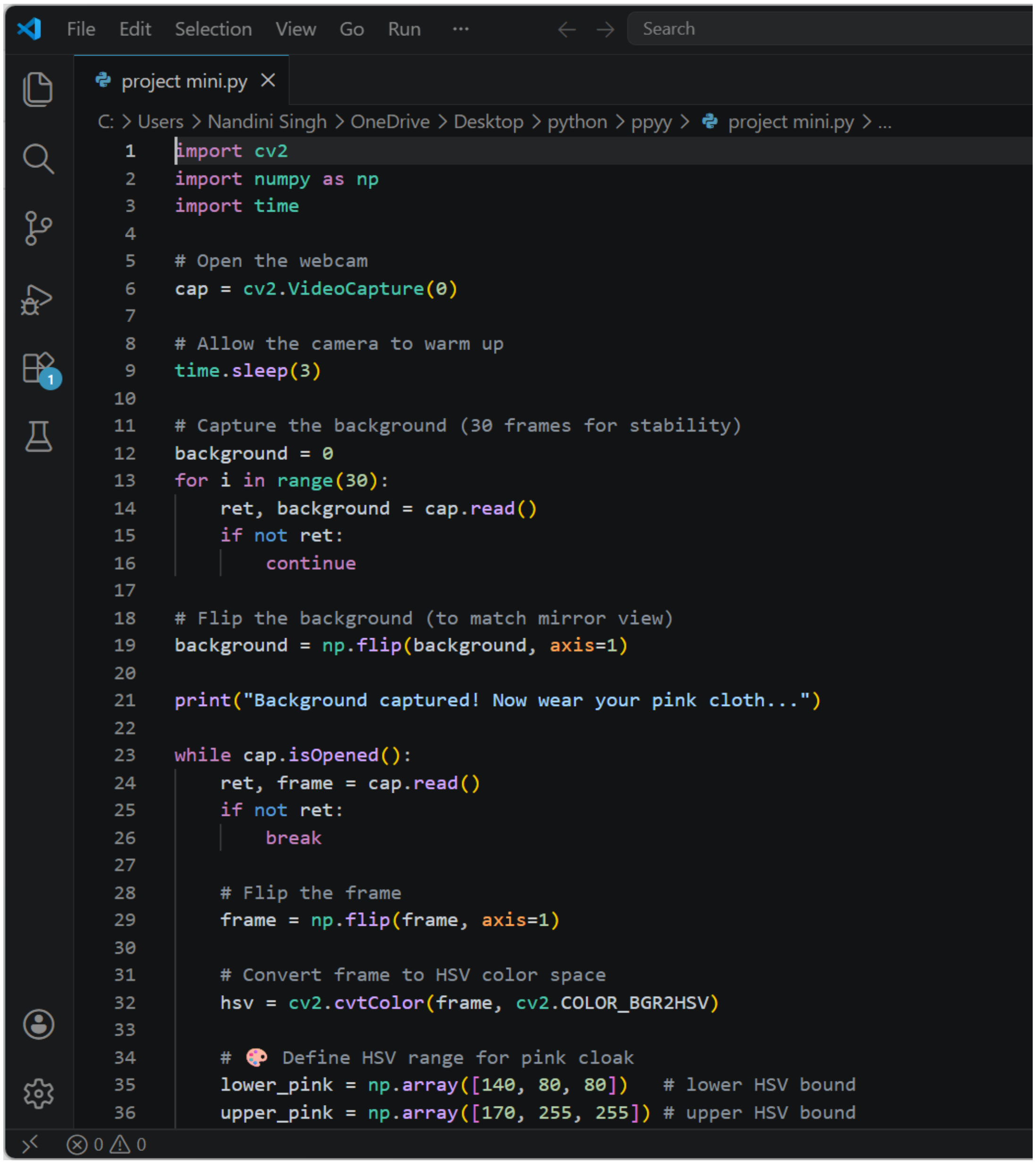
7. Background Replacement Module

Replaces the detected cloak region with the stored background image.

8. Display Module

Shows the final processed output in real time.

5.3 Snapshots of System




```python
import cv2
import numpy as np
import time

# Open the webcam
cap = cv2.VideoCapture(0)

# Allow the camera to warm up
time.sleep(3)

# Capture the background (30 frames for stability)
background = 0
for i in range(30):
    ret, background = cap.read()
    if not ret:
        continue

# Flip the background (to match mirror view)
background = np.flip(background, axis=1)

print("Background captured! Now wear your pink cloth...")

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # Flip the frame
    frame = np.flip(frame, axis=1)

    # Convert frame to HSV color space
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    # 🎨 Define HSV range for pink cloak
    lower_pink = np.array([140, 80, 80])   # lower HSV bound
    upper_pink = np.array([170, 255, 255]) # upper HSV bound
```

Figure 5.1User Interface – Live Camera Feed

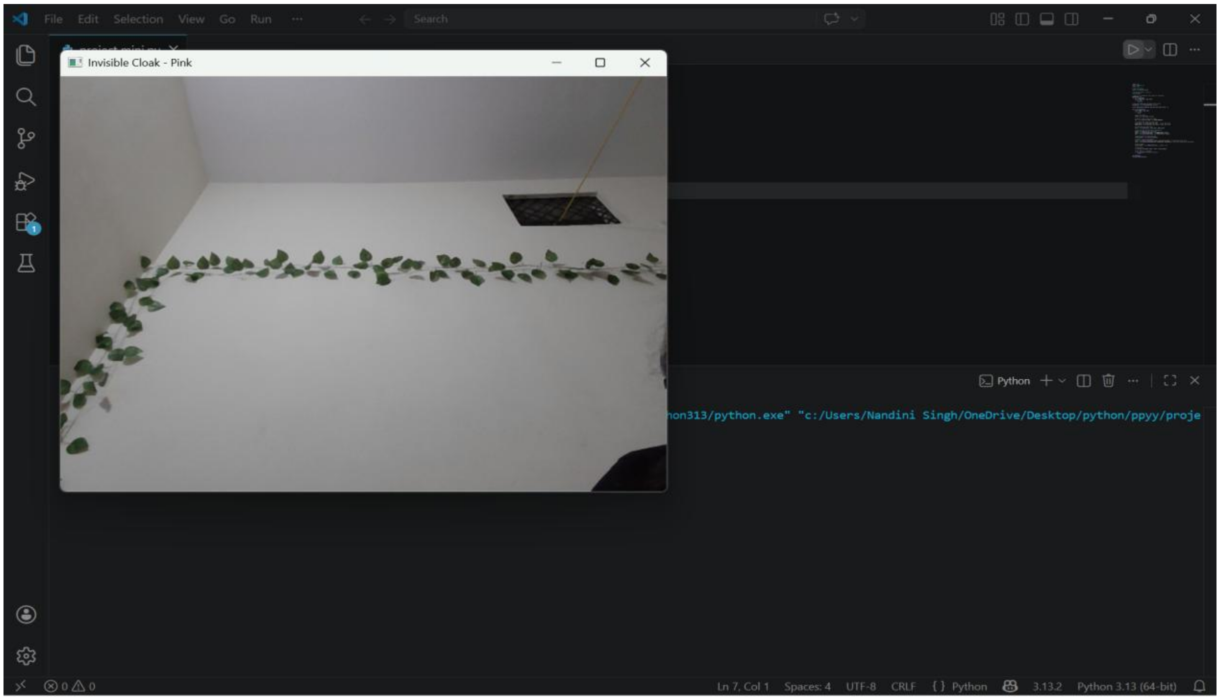


Figure 5.2 Processed Output Background Detection

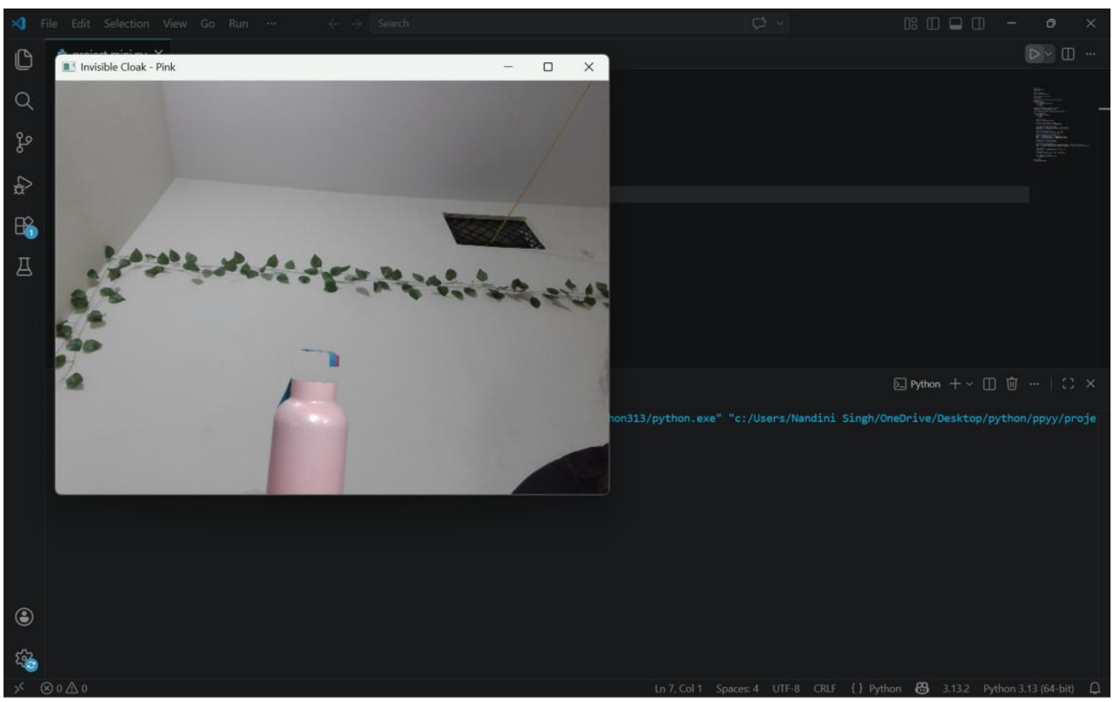


Figure 5.3 Final Background with Invisibility Effect

Chapter-6

6.1 Summary

The Invisible Cloak Project is a real-time computer vision application developed using Python and OpenCV. The primary objective of the project was to create an illusion of invisibility by detecting a specific color (pink) in a video stream and replacing it with a pre-captured background image.

The system captures live video using a webcam and processes each frame through multiple stages, including color space conversion, color detection, mask generation, and background replacement. The use of the HSV color space improves the accuracy of color detection under varying lighting conditions. Morphological operations are applied to reduce noise and enhance the quality of the mask.

The project follows a structured design approach, dividing the system into modules such as input capture, processing, and output display. Various diagrams such as Use Case Diagram, Data Flow Diagram, Activity Diagram, and Flowchart were used to represent the system architecture and workflow.

Testing was conducted at different levels, including unit testing, integration testing, and system testing, to ensure that all components function correctly and efficiently. The results demonstrate that the system performs well under controlled lighting conditions and successfully creates the invisibility effect in real time.

Overall, the project provides a practical implementation of fundamental computer vision concepts and demonstrates their application in real-world scenarios.

6.2 Conclusions

The Invisible Cloak Project successfully achieves its objective of creating a real-time invisibility effect using computer vision techniques. The system is capable of detecting a specific color and replacing it with a background image, making the object appear invisible to the viewer.

The project highlights the effectiveness of using HSV color space for color detection and demonstrates how masking and image processing techniques can be applied to manipulate visual data. The modular design ensures that each component performs a specific function, contributing to the overall efficiency of the system.

However, the system has certain limitations. Its performance depends heavily on lighting conditions, and it works best when the background remains static. Additionally, the system is limited to detecting one color at a time, which restricts its flexibility.

Despite these limitations, the project serves as a strong foundation for further development. It opens opportunities for enhancements such as multi-color detection, improved robustness under varying lighting conditions, and integration with graphical user interfaces.

In conclusion, the Invisible Cloak System is a successful implementation of real-time image processing techniques and demonstrates the potential of computer vision in creating interactive and visually appealing applications.

Chapter-7

7.1 Future Scope

The Invisible Cloak Project provides a strong foundation for further enhancements in the field of computer vision. Future improvements can focus on increasing accuracy, flexibility, and usability.

- **Multi-Color Detection:**
  Extend the system to detect and process multiple colors simultaneously instead of a single color.
- **Dynamic Background Handling:**
  Improve the system to work with moving or changing backgrounds instead of a static one.
- **Lighting Adaptation:**
  Implement adaptive algorithms to handle varying lighting conditions and reduce errors.
- **Integration with GUI:**
  Develop a graphical user interface for better user interaction and ease of use.
- **Mobile/Web Integration:**
  Convert the application into a mobile or web-based solution.
- **Augmented Reality Applications:**
  Extend the concept for use in AR systems, gaming, and entertainment industries.

7.2 Appendix

Appendix A: Sample Code Snippet

```
import cv2

import numpy as np

cap = cv2.VideoCapture(0)

# Capture background

for i in range(30):

  ret, background = cap.read()

while True:

   ret, frame = cap.read()

   hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
```

```
    lower_pink = np.array([140, 50, 50])
    upper_pink = np.array([170, 255, 255])
    mask = cv2.inRange(hsv, lower_pink, upper_pink)
    result = np.where(mask[:, :, None] == 255, background, frame)
    cv2.imshow("Invisible Cloak", result)
    if cv2.waitKey(1) == 27:
        break
cap.release()
cv2.destroyAllWindows()
```

Appendix B: Installation Commands

```
pip install opencv-python
pip install numpy
```

7.3 Bibliography

1. OpenCV Documentation
   https://docs.opencv.org/
2. Python Documentation
   https://docs.python.org/3/
3. Learning OpenCV 3
   O'Reilly Media
4. Digital Image Processing – Rafael C. Gonzalez & Richard E. Woods

7.4 List of Publications

Not applicable.

7.5 Reprints of Publications

Not applicable.